

A Systematic Mapping Study of Learned Indexes in Database Systems

Soutenance M1 MIAGE — édition publique

Seyed Amineddin Miri

Université Paris Nanterre

Entreprise : Agiga

Fonction : Full Stack Developer



CONTENU MASQUÉ / REDACTED

Le contenu de cette diapositive concerne mon employeur.

Il a été retiré de la version publique pour préserver la confidentialité des informations de l'entreprise.

CONTENU MASQUÉ / REDACTED

Le contenu de cette diapositive concerne mon employeur.

Il a été retiré de la version publique pour préserver la confidentialité des informations de l'entreprise.

CONTENU MASQUÉ / REDACTED

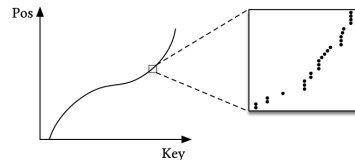
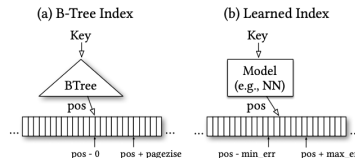
Le contenu de cette diapositive concerne mon employeur.

Il a été retiré de la version publique pour préserver la confidentialité des informations de l'entreprise.

- ▶ Un index évite de scanner toute une table. Les structures classiques (B+-trees, hash, radix) sont robustes, mais leur efficacité dépend du contexte.
- ▶ Idée : un index peut être vu comme une fonction.
- ▶ Pour des données triées, on apprend la relation :

clé $\rightarrow$ position approximative

- ▶ Le modèle prédit une position.
- ▶ Une étape de correction peut être utilisée pour garantir le résultat exact.



Figures : Kraska et al., *The Case for Learned Index Structures* (2018).

Index classique

- ▶ Structure générale et robuste.
- ▶ Moins sensible à la distribution des données.
- ▶ Très bon comportement dans beaucoup de contextes.

Learned index

- ▶ Exploite la distribution des données.
- ▶ Peut être plus compact ou plus rapide quand la distribution et le workload s'y prêtent.
- ▶ Peut nécessiter une étape de correction et du tuning.

Les deux approches sont utiles : la vraie question n'est pas *laquelle remplace l'autre*, mais *dans quels contextes* le learned index devient intéressant.

- Domaine récent (papier fondateur en 2018), déjà très hétérogène : les papiers ne comparent ni les mêmes structures, ni les mêmes métriques, ni les mêmes contextes.

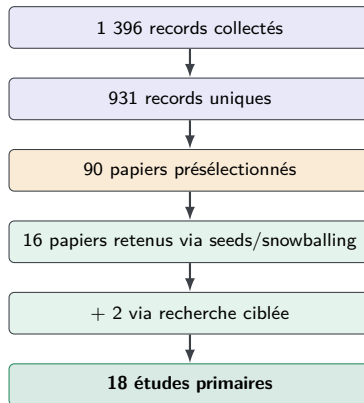
	SLR (revue systématique)	SMS (cartographie)
Question	« Quelle approche est la meilleure ? »	« Qu'a-t-on étudié, et que manque-t-il ? »
Suppose	des résultats agrégeables	accepte des études hétérogènes
Produit	un verdict	une carte : familles couvertes + zones peu étudiées
Adaptée à	un champ mûr	un champ jeune

Des surveys existent déjà, mais restent souvent très larges ; ce mémoire se distingue par des questions de recherche plus ciblées et un protocole explicite de sélection et de classification.

Le learned index remplace-t-il l'index classique, ou son gain est-il seulement conditionnel ?

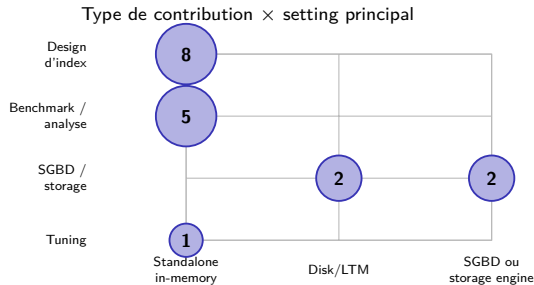
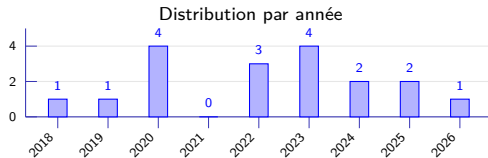
Questions de recherche

- ▶ **RQ1** — Quels types d'études sur les learned indexes ont été publiés, et comment se répartissent-ils dans le temps et selon les lieux de publication ?
- ▶ **RQ2** — Quelles techniques de learned indexes et quels scénarios de workload sont traités dans la littérature ?
- ▶ **RQ3** — Comment les learned indexes sont-ils évalués, et quelles preuves a-t-on sur la performance, la robustesse et la reproductibilité ?



Je terminerai par trois résultats concrets du corpus pour l'illustrer.

- ▶ 2018–2019 : idée fondatrice et premières structures.
- ▶ 2020–2022 : index dynamiques, benchmarks, analyses critiques.
- ▶ 2023–2026 : robustesse, disque, tuning, intégration dans PostgreSQL/LeanStore.



Couverture forte

- ▶ Indexation unidimensionnelle.
- ▶ Clés ordonnées.
- ▶ Point lookup et range queries.
- ▶ Workloads read-heavy.
- ▶ Exécution en mémoire.

Dimension	Fort	Moyen	Faible
Lookup / range			
Insertions / workloads mixtes			
Robustesse			
Concurrence			
Disque			
Intégration SGBD			

Les mises à jour sont de plus en plus étudiées ; les preuves restent plus faibles quand on se rapproche d'un SGBD complet.

Exemples : lookup/range (SOSD, ALEX, PGM) ; mises à jour (ALEX, SWIX, RoBin) ; SGBD/storage engine (PostLearn, LearnedStore).

Évaluation surtout expérimentale et de plus en plus **comparative**. Mais une même structure donne des chiffres difficiles à aligner d'un papier à l'autre :

Ce qui rend comparable

- ▶ datasets SOSD communs
- ▶ baselines fortes (B+-tree, ART...)
- ▶ code et benchmarks publics

vs

Ce qui empêche de généraliser

- ▶ workload : lookup / insert / mixte
- ▶ hardware
- ▶ tuning
- ▶ intégration (standalone ou SGBD)

→ Assez pour voir des **tendances**, pas assez pour un **verdict**.
SOSD ne trouve pas de métrique unique qui explique la performance ;
dans leur analyse, les **cache misses** sont le facteur le plus explicatif.

Évidence la plus solide

Évidence la plus fragile

plus le contexte se rapproche d'un SGBD complet, plus le gain devient incertain

- ▶ exécution en mémoire
- ▶ clés ordonnées
- ▶ workloads read-heavy
- ▶ distribution modélisable

- ▶ mises à jour lourdes
- ▶ concurrence
- ▶ stockage disque
- ▶ intégration SGBD

Le gain est réel mais **conditionnel** : il dépend de l'adéquation entre les hypothèses du modèle et le contexte système.

PostLearn — intégration dans PostgreSQL

Intégré dans PostgreSQL, le learned index perd de son avantage. Sur les grandes plages, le B+-tree gagne, car lire des pages denses est plus rapide que parcourir les gapped arrays d'ALEX+ avec vérification bitmap.

LearnedStore — modèle + fallback

Le modèle accélère la recherche dans le B+-tree de LeanStore, mais le chemin B+-tree classique reste comme fallback. Un accélérateur, pas un remplaçant.

Maltry & Dittrich — précision $\neq$ vitesse

Un modèle plus précis n'est pas automatiquement plus rapide : si l'évaluation du modèle ou la recherche de correction coûtent cher, le gain disparaît.

Conclusion

- ▶ Champ actif et en maturation.
- ▶ Preuves fortes pour certains cas en mémoire.
- ▶ Preuves plus faibles pour les cas système complets.
- ▶ Défi futur : évaluation robuste, reproductible et intégrée.

Limites du travail

- ▶ Sélection et extraction par un seul reviewer.
- ▶ Recherche ciblée limitée à l'intégration SGBD et aux moteurs de stockage.
- ▶ Artefacts (code, données) : disponibilité inégale et parfois incertaine.

Les learned indexes sont des composants dépendants du contexte, pas des remplacements universels des index classiques.

Merci de votre attention

Avez-vous des questions ?

